Likelihood II – When Closed Forms Fail

Math 742

2/3/2026

Introduction

Introduction

Recall that the basic idea of maximum likelihood estimation (MLE) is to find the parameter value that makes the observed data most plausible under the assumed model.

Likelihood function.

Given data $\mathbf{x} = (x_1, \ldots, x_n)$ and a parameter vector $\boldsymbol{\theta} \in \mathbb{R}^k$, the likelihood function is

$$L(\boldsymbol{\theta} \mid \mathbf{x}) = \prod_{i=1}^{n} f(x_i \mid \boldsymbol{\theta}),$$

Note: Unless stated otherwise, θ denotes a vector-valued parameter and derivatives are understood as gradients.

and the log-likelihood is

$$\ell(\boldsymbol{\theta}) = \sum_{i=1}^{n} \log f(x_i \mid \boldsymbol{\theta}).$$

Score function.

The score function is the gradient of the log-likelihood with respect to the parameter:

$$U(\boldsymbol{\theta}) = \nabla_{\boldsymbol{\theta}} \ell(\boldsymbol{\theta}) = \sum_{i=1}^{n} \nabla_{\boldsymbol{\theta}} \log f(x_i \mid \boldsymbol{\theta}).$$

Interior MLEs satisfy the score equation $U(\boldsymbol{\theta}) = \mathbf{0}$, but maximizers may also occur on the boundary of the parameter space or at points where the likelihood is not differentiable.

Solving the Score Equation

For many models, the score equation

$$U(\boldsymbol{\theta}) = \mathbf{0}$$

is a nonlinear system of equations that cannot be solved analytically. As a result, computing the MLE is fundamentally a *numerical root-finding problem.*

Newton-Raphson Algorithm

The Newton–Raphson method is a general-purpose iterative algorithm for solving nonlinear equations. Applied to maximum likelihood estimation, it updates the parameter by repeatedly solving a local quadratic approximation to the log-likelihood. Newton–Raphson uses the observed Hessian → quadratic approximation to the actual log-likelihood surface at the current point.

Before we go into the multidimensional version, for understanding I start with the 1D version.

- $\ell'(\theta)$, slope (first derivative)
- $\ell''(\theta)$, curvature (second derivative)

$$\boxed{\theta_{k+1} = \theta_k - \frac{\ell'(\theta)}{\ell''(\theta)}}$$

This equation comes by approximating the quadratic approximation (Taylor Series) of the log likelihood function near the maximum.

$$\ell(\theta) \approx \ell(\theta_k) + \ell'(\theta_k)(\theta - \theta_k) + \frac{1}{2}\ell''(\theta_k)(\theta - \theta_k)^2$$

If you take the derivative of this approximate equation, you get the Newton-Raphson function in the box above.

Essentially, pretend the log-likelihood is a parabola right here and jump to the top of that parabola.

For multiple dimensions, let

$$U(\boldsymbol{\theta}) = \nabla_{\boldsymbol{\theta}}\ell(\boldsymbol{\theta}), \qquad H(\boldsymbol{\theta}) = \nabla^2_{\boldsymbol{\theta}}\ell(\boldsymbol{\theta})$$

denote the score vector and Hessian matrix, respectively.

Starting from an initial guess $\boldsymbol{\theta}^{(0)}$, the Newton–Raphson update is

$$\boxed{\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - [H(\boldsymbol{\theta}_t)]^{-1} U(\boldsymbol{\theta}_t).}$$

At each iteration, Newton–Raphson approximates the log-likelihood by a quadratic function and jumps to the maximizer of that quadratic approximation.

- $\nabla\ell$ points "uphill"
- H^{-1} rescales and rotates that direction based on the curvature

As we will learn, this connects beautifully to Fisher information:

$$\mathcal{I}(\theta) = -\mathbb{E}[H(\theta)]$$

- Expected curvature is often smoother,
- Guarantees positive definiteness in many models,
- Often more stable early in iterations.

Warning: Newton–Raphson with observed Hessian can be unstable or even go in the wrong direction when far from the MLE (especially when the observed information is not positive definite).

Fisher Scoring

A closely related method is *Fisher scoring*, which replaces the observed Hessian $H(\boldsymbol{\theta})$ with its expectation, the Fisher information matrix. The advantage of Fisher scoring, is that it guarantees that the update direction is an ascent direction under mild conditions (because expected information is positive semi-definite under regularity).

Newton-Raphson uses the observed Hessian ($H(\theta_k)$), Fisher scoring replaces it with the expected Hessian: $\mathcal{I}(\theta) = -\mathbb{E}[H(\theta)]$

Unfortunately, many likelihood functions lead to nonlinear score equations that do not admit closed-form solutions. Fisher scoring is a variant of the Newton–Raphson algorithm in which the observed information (the Hessian of the log-likelihood) is replaced by its expectation, the Fisher information. Examples include:

- Logistic regression
- Gamma, Weibull, Beta, and negative binomial models with unknown shape or dispersion
- Generalized linear models with multiple predictors
- Mixed-effects models, zero-inflated models, and survival models

As a result, most MLEs encountered in practice are obtained using iterative numerical optimization algorithms rather than by solving the score equations analytically. In fact, most of the time we have to use numerical optimization because closed form solutions do not exist.

Example: Suppose we want to determine the MLE for the logistics regression model. In other words, what is the vector of coefficients β that is most likely given the data.

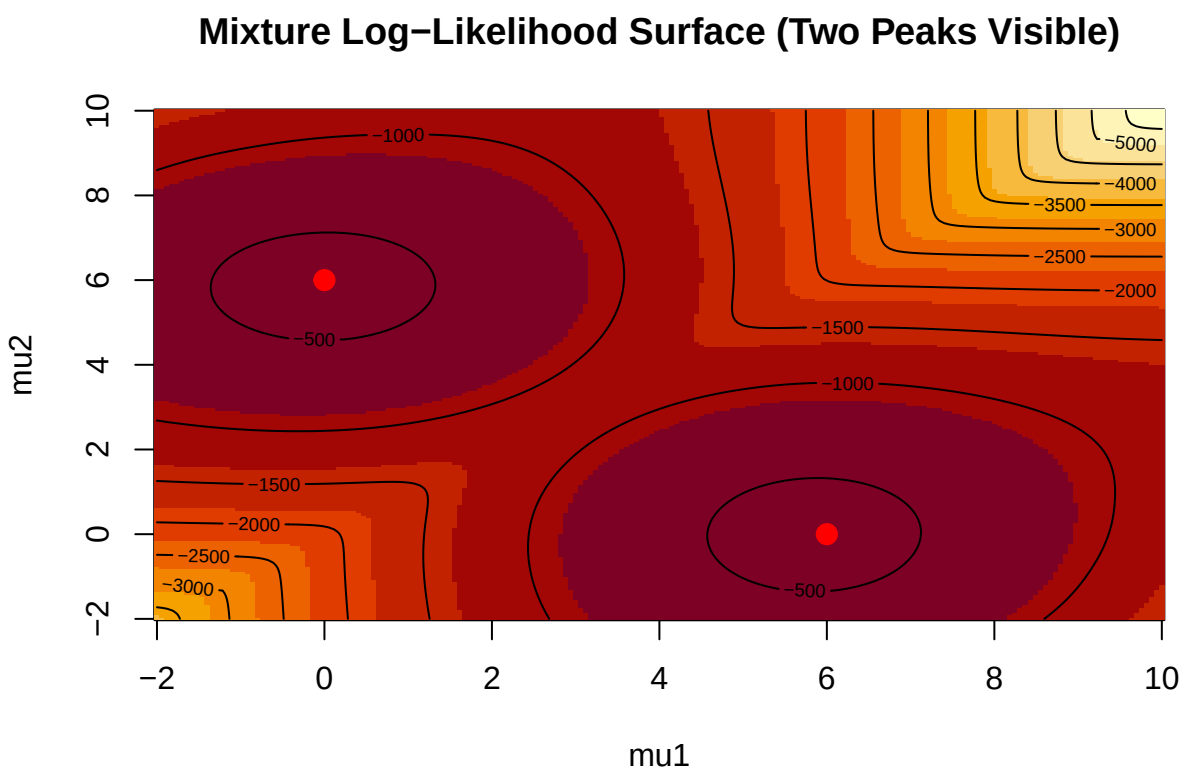
$$\text{logit}\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 X_1 + ... + X_p\beta_p$$

The log likelihood function is: $\ell(\beta) = \sum_{i=1}^{n}\left[y_i x_i^\top \beta - \log(1 + e^{x_i^\top \beta})\right]$

- Gradient: nonlinear in β
- Hessian: depends on fitted probabilities
- No closed form

The Hessian measures how curved the log-likelihood is; Newton–Raphson uses that curvature to choose a step size that jumps toward the maximum instead of blindly walking uphill.

Consider the following log-likelihood surface of a distribution comprised of the mixture of two Normal distributions. Here we see two symmetric peaks, one near (0,6) and the second near (6,0). Between the two peaks you see a long ridge of nearly constant likelihood, many parameter combinations will fit almost equally well. This means the likelihood surface is non-convex, has multiple local maxima, and provides no closed-form solution.



When Closed Forms Don't Exist

There are various numerical optimization techniques that we can use:

- Newton–Raphson uses observed curvature
- Fisher scoring uses expected curvature

→ Both are second-order methods

→ Fisher scoring is often more stable early in optimization

→ Gradient-Free Methods

- Nelder–Mead
- Grid search

→ Practical Considerations

- Starting values
- Convergence diagnostics
- Boundary issues

→ In non-convex likelihoods (e.g. mixtures), numerical optimization may converge to local maxima—multiple starts are essential.

Rather than focusing on how the algorithms work, I will take more of a tools approach. Here are some tools for finding the MLE when no closed form solution exists.

R tools

- optim()

- nlm() / nlminb()
- Model-specific high-level functions:

  glm(), glmnet(), lme4::glmer(), mgcv, survival models, etc.
- Special case optimizers:

  uniroot() for 1-D likelihood equations

  optimx::optimx() for multi-algorithm comparison
- Constraints: using optim(..., method="L-BFGS-B") and reparameterization tricks

To show you how this works, I will actually use an example where there is a closed-form solution. So we can compare the optimization result with the true MLE estimator.

```
set.seed(2718)

# Simulate Poisson data
n <- 200
lambda_true <- 3
x <- rpois(n, lambda_true)

# Negative log-likelihood
neg_loglik <- function(lambda, x) {
  if (lambda <= 0) return(Inf)     # enforce positivity
  -sum(dpois(x, lambda, log = TRUE))
}

# Use optim() to minimize negative log-likelihood
fit <- optim(
  par = 1,                    # initial guess
  fn = neg_loglik,
  x = x,
  method = "L-BFGS-B",
  lower = 1e-6,
  upper = 100
)

fit$par           # Estimated lambda
```

```
## [1] 3.03
```

```
mean(x)           # True analytic MLE for comparison
```

```
## [1] 3.03
```

Perfect match.

Python Tools

- scipy.optimize.minimize()
- scipy.optimize.root()
- statsmodels (logit, GLM, MLEBase if you want a "build your own MLE")
- jax ("show them the future"): autodiff + autodiff-validated Newton steps
- pytorch for autodiff-based custom likelihoods

Most real-world MLEs have no closed-form solution.
We find them with iterative algorithms (`optim()`, `glm()`, etc.).

These algorithms can fail on multimodal or flat likelihood surfaces
→ always check starting values and convergence!

Invariance Property of the MLE (Review)

Core Idea: If $\hat{\theta}$ is the MLE of θ, then $\hat{\phi} = g(\hat{\theta})$ is the MLE of $g(\theta)$. Basically, the invariance property of MLEs is that the MLE of a function of a parameter is that same function applied to the MLE of the original parameter.

Example:

Let

$$X_1, \ldots, X_n \overset{iid}{\sim} \text{Bernoulli}(p),$$

and recall that the MLE of p is

$$\hat{p} = \bar{X}.$$

Define the *odds* parameter

$$\theta = g(p) = \frac{p}{1-p}, \qquad p \in (0, 1).$$

Since g is a one-to-one transformation, the invariance property implies that the MLE of θ is obtained by transforming $\hat{p}$:

$$\hat{\theta} = g(\hat{p}) = \frac{\hat{p}}{1-\hat{p}} = \frac{\bar{X}}{1-\bar{X}}.$$

No re-optimization of the likelihood is required; the MLE of the transformed parameter is obtained by direct substitution.

We can simulate this example numerically.

1. Randomly sample from a binomial distribution with known p.
2. Estimate the p via MLE.
3. Apply the logit transformation:

$$\hat{\phi} = \log\left(\frac{p}{1-p}\right)$$

Reparameterization changes the shape of the optimization landscape but not the underlying statistical model.

4. Show that $\hat{\phi} = g(\hat{\theta})$ is the MLE of $g(\theta)$.

```r
set.seed(9876)        # for reproducibility

n <- 20
true_p <- 0.35
y <- rbinom(1, size = n, prob = true_p)

cat(sprintf("y = %d out of n = %d\n", y, n))
```

```
## y = 9 out of n = 20
```

```r
cat(sprintf("True p = %.3f\n\n", true_p))
```

```
## True p = 0.350
```

```r
# Direct MLE
phat <- y / n
cat(sprintf("Direct MLE phat = %.6f\n", phat))
```

```
## Direct MLE phat = 0.450000
```

```r
# Logit transformation functions
p_to_phi <- function(p) log(p / (1 - p))
phi_to_p <- function(phi) 1 / (1 + exp(-phi))

# Log-likelihood in phi-space
loglik_phi <- function(phi, y, n) {
  p <- phi_to_p(phi)
  y * log(p) + (n - y) * log(1 - p)
}

# Numerical optimization
opt <- optim(par = 0,
             fn = function(phi) -loglik_phi(phi, y, n),
             method = "BFGS")

phi_num <- opt$par                              # from optim
phi_exact <- p_to_phi(phat)                     # exact analytic

cat(sprintf("phi from optim   : %.10f\n", phi_num))
```

```
## phi from optim   : -0.2006707191
```

```r
cat(sprintf("phi exact formula: %.10f\n", phi_exact))
```

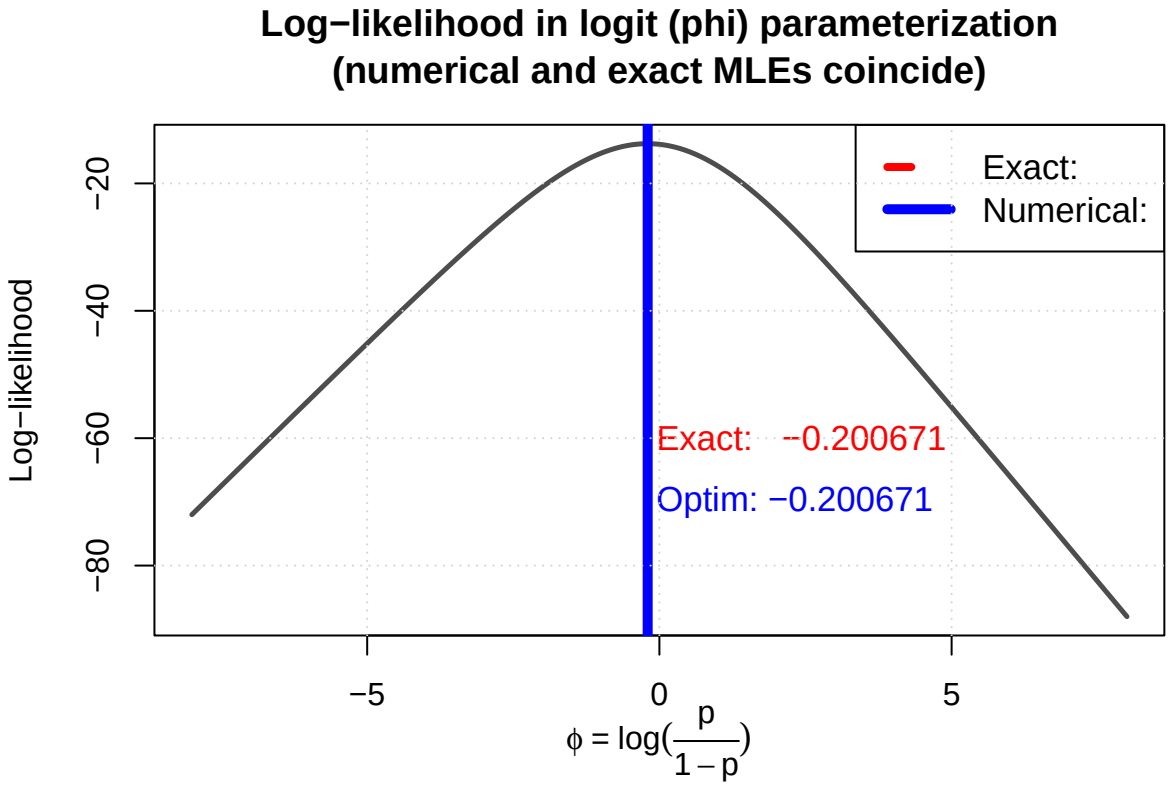
```
## phi exact formula: -0.2006706955
```

```r
cat(sprintf("Difference       : %.2e\n\n", abs(phi_num - phi_exact)))
```

```
## Difference       : 2.36e-08
```

```r
phi_grid <- seq(-8, 8, length = 2000)
ll_phi   <- sapply(phi_grid, loglik_phi, y = y, n = n)

plot(phi_grid, ll_phi, type = "l", lwd = 2.5, col = "gray30",
     xlab = expression(phi ~ "=" ~ log(frac(p, 1-p))),
     ylab = "Log-likelihood",
     main = "Log-likelihood in logit (phi) parameterization\n(numerical and exact MLEs coincide)")
```

```r
# Exact analytic result (red, dashed, slightly thicker)
abline(v = phi_exact, col = "red", lwd = 4, lty = 2)

# Numerical optimization result (blue, solid, even thicker so it's on top)
abline(v = phi_num, col = "blue", lwd = 5, lty = 1)

# Add a subtle offset label so you can see both values
text(phi_exact + 0.15, -60,
     sprintf("Exact:   %.6f", phi_exact), col = "red",  adj = 0, cex = 1.1)
text(phi_exact + 0.15, -70,
     sprintf("Optim: %.6f", phi_num),   col = "blue", adj = 0, cex = 1.1)

legend("topright",
       legend = c("Exact:", "Numerical:"),
       col = c("red", "blue"),
       lwd = c(4, 5), lty = c(2, 1), cex = 1.1)

grid()
```



When MLE Optimization Can Fail

- Poor starting values
- Flat likelihoods
- Multiple local maxima
- Boundary solutions
- Non-identifiability. A model is non-identifiable if different parameter values produce the same probability

distribution for the data.

→ Always check convergence diagnostics and try multiple starting points.

Summary

- The usual MLE for p is $\hat{p} = y/n$.
- Using the logit transformation ($\hat{\phi}$), maximizing this with respect to *phi* recovers $\hat{\phi} = \log\left(\frac{p}{1-p}\right)$

Examples:

- Estimating σ from the normal likelihood
- Estimating rate vs. mean for an exponential distribution
- Estimating odds ratio from logistic regression coefficients

> **If $\hat{\theta}$ is the MLE of θ, then $g(\hat{\theta})$ is automatically the MLE of $g(\theta)$** — no matter how nonlinear $g(\cdot)$ is.
>
> The maximizer transforms exactly when you reparameterize the likelihood.

Profile Likelihood

Core Idea: Suppose the parameter vector can be written as $\theta = (\psi, \lambda)$, where ψ is the parameter of interest and λ is a nuisance parameter.

The profile log-likelihood for ψ is defined as

$$\ell_p(\psi) = \max_{\lambda} \ell(\psi, \lambda).$$

When estimating a model, we often care about only one parameter (such as a mean), even though the model contains additional parameters (such as a variance).
Profile likelihood provides a way to focus on ψ while still accounting for the uncertainty in the other parameters.
To compute the profile likelihood, we fix ψ at a candidate value, then choose the value of λ that maximizes the log-likelihood for that fixed ψ.
Repeating this process over a range of values of ψ produces the profile log-likelihood curve.

Example: Normal distribution with unknown mean μ and variance σ^2.

- For each fixed μ on a grid, maximize likelihood over σ^2 using an optimizer.
- Plot the resulting profile likelihood curve.
- Overlay the quadratic approximation (Taylor expansion) so they see how much better profile likelihood can behave.

```
# ================================================================
# Profile Likelihood Simulation for Normal Data
# N(mu, sigma^2) - both parameters unknown
# Shows: profile log-likelihood vs quadratic (normal) approximation
# ================================================================

set.seed(11252025)
n <- 50
true_mu <- 3.0
true_sigma <- 1.5

# Simulate data
```

```r
x <- rnorm(n, mean = true_mu, sd = true_sigma)

# Sample mean and variance (for reference)
xbar <- mean(x)
s2   <- var(x)           # sample variance
cat(sprintf("n = %d,  xbar = %.3f,  s^2 = %.3f\n", n, xbar, s2))
```

```
## n = 50,  xbar = 3.080,  s^2 = 1.706
```

```r
cat(sprintf("True mu = %.2f, true sigma = %.2f\n\n", true_mu, true_sigma))
```

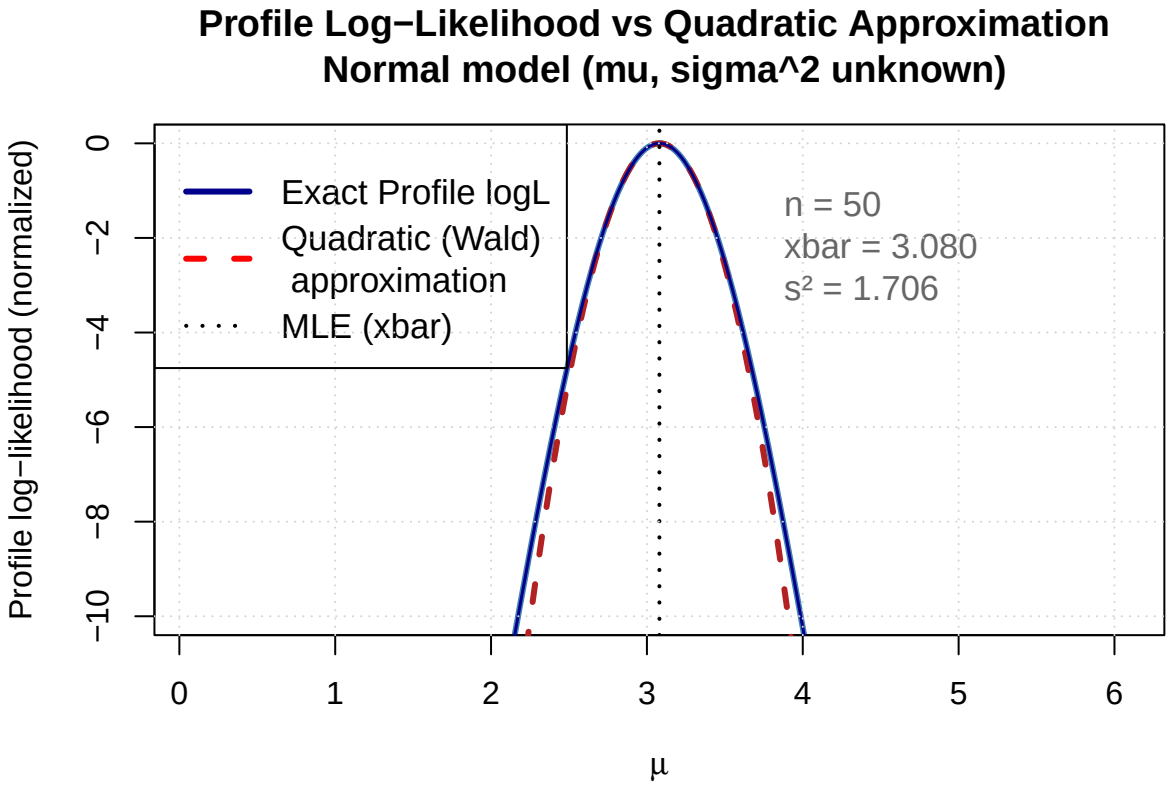
```
## True mu = 3.00, true sigma = 1.50
```

```r
# Full log-likelihood function l(mu, sigma^2)
loglik_full <- function(mu, sig2, x) {
  n <- length(x)
  -n/2 * log(2*pi) - n/2 * log(sig2) - sum((x - mu)^2)/(2*sig2)
}

# For a FIXED mu, maximize over sigma^2 analytically
# We know: sigma^2_hat(mu) = (1/n) * sum(x_i - mu)^2
sigma2_hat <- function(mu, x) mean((x - mu)^2)

# Profile log-likelihood: plug in the conditional MLE for sigma^2
profile_loglik <- function(mu, x) {
  sig2 <- sigma2_hat(mu, x)
  loglik_full(mu, sig2, x)
}

# ------------------------------------------------------------------
# 1. Evaluate profile log-likelihood on a fine grid of mu
# ------------------------------------------------------------------
mu_grid <- seq(xbar - 3, xbar + 3, length.out = 800)

prof_ll <- sapply(mu_grid, profile_loglik, x = x)

# Find the profile MLE (should be exactly xbar!)
mu_prof_mle <- mu_grid[which.max(prof_ll)]
cat(sprintf("Profile MLE for mu: %.6f  (sample mean = %.6f)\n\n", mu_prof_mle, xbar))
```

```
## Profile MLE for mu: 3.076342  (sample mean = 3.080097)
```

```r
# ------------------------------------------------------------------
# 2. Quadratic (Taylor) approximation around the MLE
# ------------------------------------------------------------------
# We know asymptotic variance of hat{mu} is sigma^2 / n
# So -2 * (l(mu) - l(max)) approx (mu - xbar)^2 / (sigma^2 / n)
# But since sigma^2 is unknown, the usual Wald quadratic uses s^2:
quadratic_approx <- function(mu, xbar, s2, n) {
  -(n/2) * (mu - xbar)^2 / s2
}

# Normalize both so they touch at the maximum (for fair visual comparison)
prof_ll_normalized      <- prof_ll - max(prof_ll)
quadratic_ll_normalized <- quadratic_approx(mu_grid, xbar, s2, n)
quadratic_ll_normalized <- quadratic_ll_normalized - max(quadratic_ll_normalized)
```

```r
# ------------------------------------------------------------------
# 3. Plot both curves
# ------------------------------------------------------------------
plot(mu_grid, prof_ll_normalized, type = "l", lwd = 3, col = "steelblue",
     ylim = c(-10, 0),
     xlab = expression(mu),
     ylab = "Profile log-likelihood (normalized)",
     main = "Profile Log-Likelihood vs Quadratic Approximation \n Normal model (mu, sigma^2 unknown)")

# Overlay the quadratic approximation
lines(mu_grid, quadratic_ll_normalized, col = "firebrick", lwd = 3, lty = 2)

# True profile likelihood is exact: - (n/2) * log(1 + n(mu - xbar)^2 / (n-1)s^2 )  [up to const]
# Let's add the EXACT profile likelihood (not just numerical)
exact_profile <- -(n/2) * log(1 + n*(mu_grid - xbar)^2 / ((n-1)*s2))
exact_profile <- exact_profile - max(exact_profile)
lines(mu_grid, exact_profile, col = "darkblue", lwd = 1.5, lty = 1)

# Vertical line at true MLE
abline(v = xbar, col = "black", lty = 3, lwd = 2)

# Legend and annotations
legend("topleft",
       legend = c("Exact Profile logL",
                  "Quadratic (Wald) \n approximation",
                  "MLE (xbar)"),
       col = c("darkblue", "red", "black"),
       lwd = c(3, 3, 2),
       lty = c(1, 2, 3),
       cex = 1.1)

text(xbar + 0.8, -1,
     sprintf("n = %d\nxbar = %.3f\ns² = %.3f", n, xbar, round(s2,3)),
     adj = c(0,1), cex = 1.1, col = "gray40")

grid()
```



Summary

The profile log-likelihood (up to a constant) is exactly

$$\ell_p(\mu) \;=\; -\frac{n}{2}\,\log\left(1+\frac{n(\mu-\bar{x})^2}{(n-1)s^2}\right) \;=\; -\frac{n}{2}\,\log\left(1+\frac{t^2}{n-1}\right)$$

where $t = \sqrt{n}\,\dfrac{\mu-\bar{x}}{s}$ is the usual one-sample t-statistic.

Therefore:

- The **shape** of the profile log-likelihood for μ is **exactly the same** as the log-density of a Student-t random variable with $n-1$ degrees of freedom (up to location/scale and an additive constant).
- This is why profile likelihood confidence intervals for the normal mean are **identical** to the classic t-based confidence intervals.
- This is also why the profile log-likelihood has **heavier tails** than the quadratic/Wald approximation.
- The quadratic approximation is good near the MLE but deviates in the tails
- Profile likelihood is much better behaved for confidence intervals!
- The true profile log-likelihood (blue) has heavier tails than the quadratic approximation (red dashed).
- The quadratic (Wald) approximation assumes normality of $\hat{\mu}$ and uses s^2, it's only accurate locally.
- The exact profile likelihood follows a scaled log-F shape (equivalent to a Student-t likelihood), which gives better coverage for confidence intervals, especially with small n.

Profile likelihood is used for:

- Confidence intervals

- Checking identifiability
- Bootstrap alternatives
- Model sanity checks

When Profile Likelihood Is Useful

- Nuisance parameters can't be eliminated analytically
- Confidence intervals from profile likelihood
- Likelihood ratio tests

→ Examples

- Normal distribution with unknown μ and σ^2: profiling
- Poisson mean with offset
- Logistic regression: profiling for a single coefficient

When nuisance parameters cannot be eliminated analytically,
profile log-likelihood $\ell_p(\psi) = \max_\lambda \ell(\psi, \lambda)$ is the gold standard.
It is exact (not quadratic), has heavier tails, and gives superior confidence intervals compared to Wald/SE-based intervals — especially with small n.

Big Picture

Likelihood is the unifying language of modern inference.
MLE → numerical optimization when closed forms fail.
Standard errors lie → use profile likelihood.
Everything else (Bayes, penalized, empirical likelihood) is just likelihood + extra ingredients.

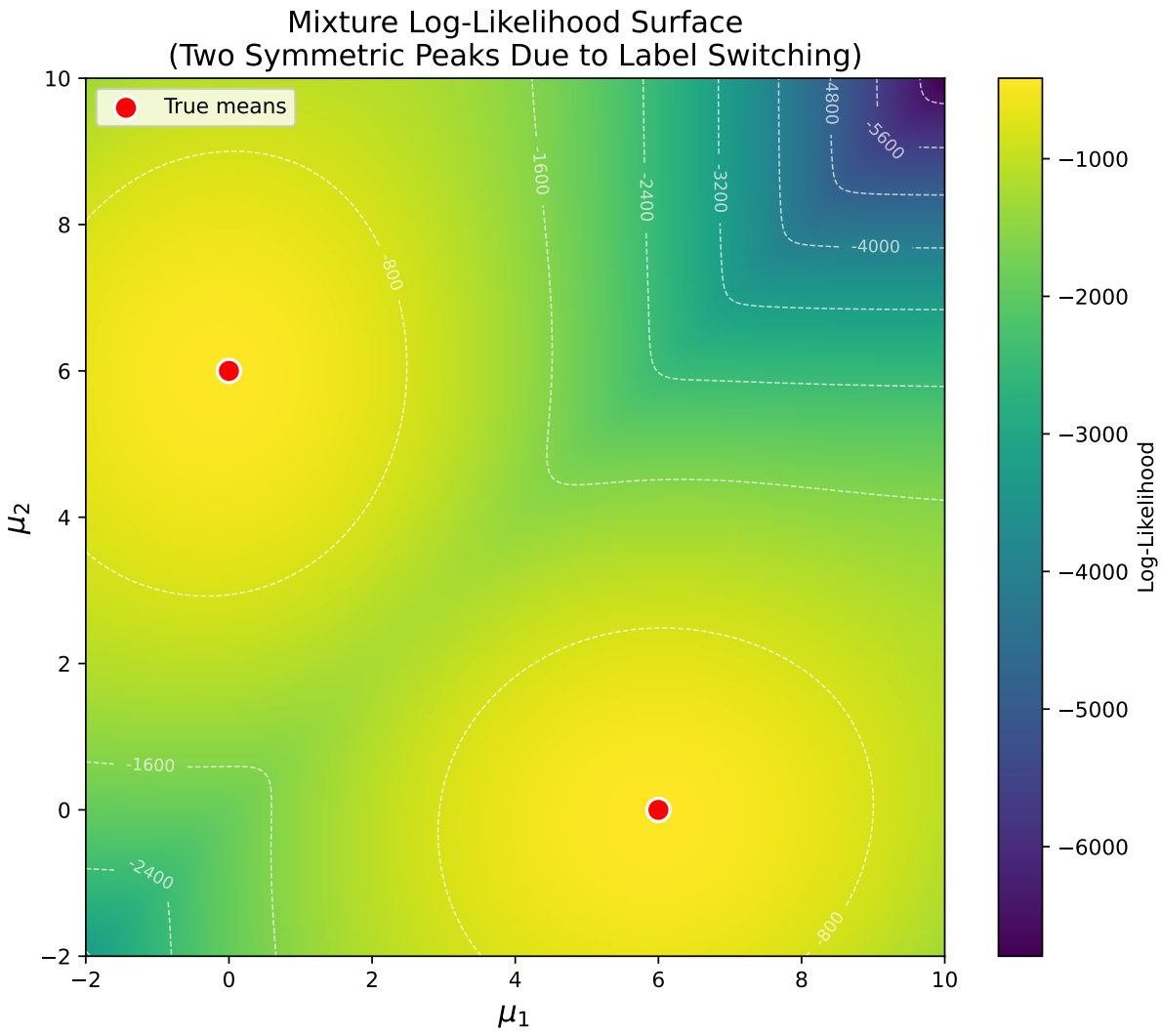
Python Code

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

np.random.seed(999)

n = 200
mu1_true, mu2_true = 0, 6
sigma = 1

z = np.random.binomial(1, 0.5, n)
x = np.where(z == 1, np.random.normal(mu1_true, sigma, n), np.random.normal(mu2_true, sigma, n))

# Fully vectorized log-likelihood
def compute_loglik_surface(x, mu_grid, sigma=1.0):
    MU1, MU2 = np.meshgrid(mu_grid, mu_grid)
    # Shape: (n_grid, n_grid, n_samples)
    logpdf1 = norm.logpdf(x, loc=MU1[..., np.newaxis], scale=sigma)
    logpdf2 = norm.logpdf(x, loc=MU2[..., np.newaxis], scale=sigma)

    # Log-sum-exp over the two components
```

```
    log_mix = np.logaddexp(logpdf1 + np.log(0.5), logpdf2 + np.log(0.5))
    LL = log_mix.sum(axis=-1)  # sum over observations
    return MU1, MU2, LL

mu_grid = np.linspace(-2, 10, 200)
MU1, MU2, LL = compute_loglik_surface(x, mu_grid, sigma)

# Plot
plt.figure(figsize=(8, 7))
plt.imshow(LL, extent=(-2, 10, -2, 10), origin='lower', cmap='viridis', aspect='auto')
plt.colorbar(label='Log-Likelihood')
```

```
## <matplotlib.colorbar.Colorbar object at 0x000002E3950A1480>
```

```
contour = plt.contour(mu_grid, mu_grid, LL, colors='white', alpha=0.7, linewidths=0.7)
plt.clabel(contour, inline=True, fontsize=8, fmt='%.0f')

# True symmetric maxima
plt.scatter([0, 6], [6, 0], color='red', s=120, edgecolor='white', linewidth=1.5, zorder=5, label='True

plt.xlabel(r'$\mu_1$', fontsize=14)
plt.ylabel(r'$\mu_2$', fontsize=14)
plt.title('Mixture Log-Likelihood Surface\n(Two Symmetric Peaks Due to Label Switching)', fontsize=14)
plt.legend()
plt.tight_layout()
plt.show()
```




```python
# Python equivalent
import numpy as np
from scipy.optimize import minimize
from scipy.stats import poisson

np.random.seed(98765)

# Simulate Poisson data
n = 200
lambda_true = 3
x = np.random.poisson(lambda_true, size=n)

# Negative log-likelihood
def neg_loglik(param):
    lmbda = param[0]
    if lmbda <= 0:
        return np.inf
    return -np.sum(poisson.logpmf(x, mu=lmbda))
```

```python
# Optimize
res = minimize(
    fun=neg_loglik,
    x0=[1.0],                  # starting value
    method='L-BFGS-B',
    bounds=[(1e-8, None)]      # enforce lambda > 0
)

print(f"Estimated lambda: {res.x[0]:.3f}")
```

```
## Estimated lambda: 3.145
```

```python
print(f"Sample mean:      {np.mean(x):.3f}")
```

```
## Sample mean:      3.145
```

```python
import numpy as np
import matplotlib
matplotlib.use('Agg', force=True)   # ← forces non-interactive backend (knitting-safe)
import matplotlib.pyplot as plt
plt.rcParams['figure.dpi'] = 150

# Reproducibility
np.random.seed(11252025)

# Simulate data
n = 50
true_mu = 3.0
true_sigma = 1.5
x = np.random.normal(loc=true_mu, scale=true_sigma, size=n)

# Sample statistics
xbar = np.mean(x)
s2 = np.var(x, ddof=1)         # unbiased variance (matches R's var())

print(f"n = {n}, xbar = {xbar:.3f}, s² = {s2:.3f}")
```

```
## n = 50, xbar = 2.644, s² = 3.474
```

```python
print(f"True mu = {true_mu:.2f}, true sigma = {true_sigma:.2f}\n")
```

```
## True mu = 3.00, true sigma = 1.50
```

```python
# Log-likelihood functions
def loglik_full(mu, sig2, x):
    n = len(x)
    return -n/2*np.log(2*np.pi) - n/2*np.log(sig2) - np.sum((x-mu)**2)/(2*sig2)

def sigma2_hat(mu, x):
    return np.mean((x-mu)**2)

def profile_loglik(mu, x):
    return loglik_full(mu, sigma2_hat(mu, x), x)

# Grid
mu_grid = np.linspace(xbar-3, xbar+3, 800)
```

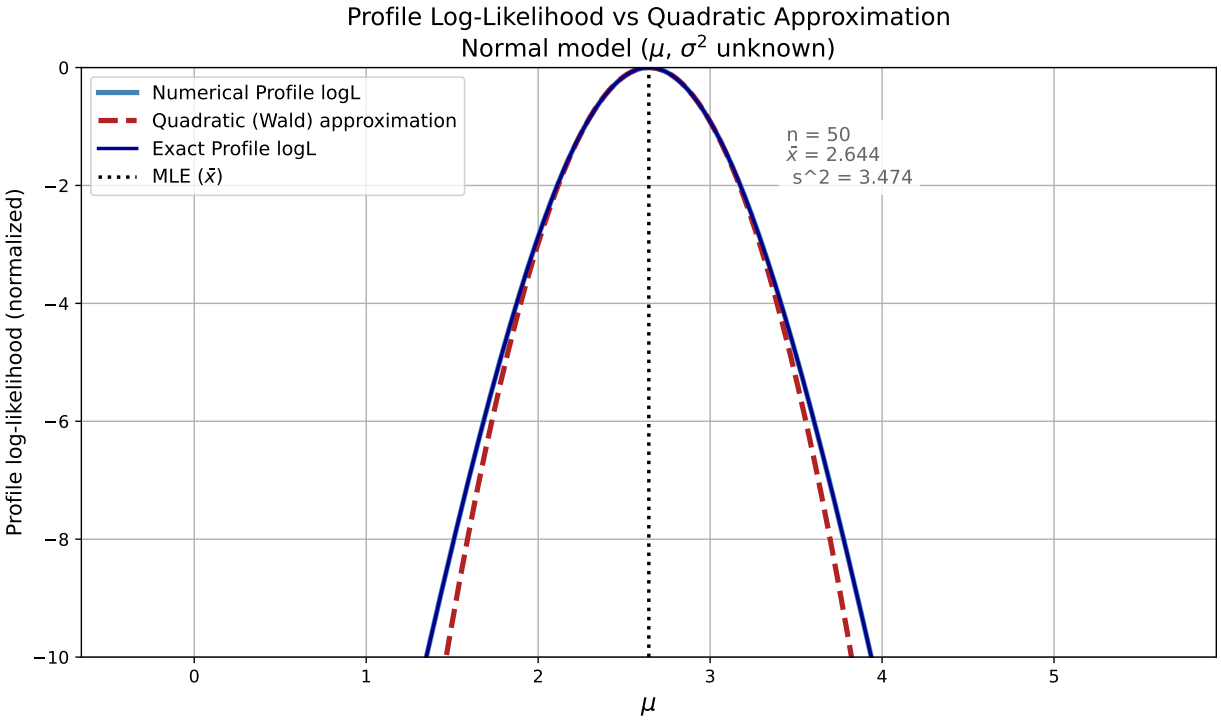
```
prof_ll = np.array([profile_loglik(m, x) for m in mu_grid])
mu_mle = mu_grid[np.argmax(prof_ll)]
print(f"Profile MLE for mu: {mu_mle:.6f} (sample mean = {xbar:.6f})\n")
```

```
## Profile MLE for mu: 2.639853 (sample mean = 2.643607)
```

```
# Normalised curves
prof_norm   = prof_ll - prof_ll.max()
quad_norm   = -(n/2)*(mu_grid-xbar)**2/s2
quad_norm   = quad_norm - quad_norm.max()
exact_norm  = -(n/2)*np.log(1 + n*(mu_grid-xbar)**2/((n-1)*s2))
exact_norm  = exact_norm - exact_norm.max()

# Plot
fig, ax = plt.subplots(figsize=(10,6))
ax.plot(mu_grid, prof_norm,   color='steelblue', lw=3,   label='Numerical Profile logL')
ax.plot(mu_grid, quad_norm,   color='firebrick', lw=3, linestyle='--', label='Quadratic (Wald) approxima
ax.plot(mu_grid, exact_norm,  color='darkblue',  lw=2,               label='Exact Profile logL')
ax.axvline(xbar, color='black', linestyle=':', lw=2, label=r'MLE ($\bar{x}$)')

ax.set_ylim(-10, 0)
```

```
## (-10.0, 0.0)
```

```
ax.set_xlabel(r'$\mu$', fontsize=14)
ax.set_ylabel('Profile log-likelihood (normalized)', fontsize=12)
ax.set_title(r'Profile Log-Likelihood vs Quadratic Approximation' '\n' r'Normal model ($\mu$, $\sigma^2$
ax.legend(loc='upper left', fontsize=11)
ax.grid(True)

ax.text(xbar + 0.8, -1,
        f'n = {n}\n$\\bar{{x}}$ = {xbar:.3f}\n s^2 = {s2:.3f}',
        ha='left', va='top', fontsize=11, color='#666666',   # ← valid dark gray
        bbox=dict(facecolor='white', alpha=0.85, edgecolor='none'))

plt.tight_layout()

plt.gcf()
```


Profile Log-Likelihood vs Quadratic Approximation
Normal model (μ, σ^2 unknown)
Numerical Profile logL
Quadratic (Wald) approximation
Exact Profile logL
MLE ($\bar{x}$)
n = 50
$\bar{x}$ = 2.644
s^2 = 3.474
Profile log-likelihood (normalized)
μ
0
−2
−4
−6
−8
−10
0
1
2
3
4
5